Edge Fog and Cloud Task Placement Simulator

ARM 305 Mini Project
Cloud Dev Edge and Fog Computing

Objective and outcome

This project implements a locally runnable Python simulator that assigns computing tasks to Edge, Fog or Cloud nodes. It models exclusive CPU and RAM reservations, a simulated VM lifecycle, response time and tenant authorization. A Streamlit interface supports task entry and inspection, while a command-line runner regenerates evidence without the interface.

The reproducible 12-task workload completes under both policies. Lowest-latency placement yields a mean response of **69.25 ms**, compared with **97.10 ms** for the capacity-aware policy. The latter spreads demand more evenly but sends more work to higher-latency nodes. Under a 30-task overload, the simulator completes 17 tasks and rejects 13. It denies an unauthorized task read and excludes the other tenant from exports.

Implemented scope

Capability	Implementation
Nodes	Edge, Fog and Cloud with configurable capacity and base latency
Workloads	Seeded generation of 10 or more tasks, plus manual task entry
Placement	Two deterministic rule-based policies with capacity and QoS checks
Virtualization	One VM per admitted task with create, complete and stop operations
Security	Tenant checks on submission, direct read, lists, VM stop and export
Evidence	Four experiments, CSV and JSON results, Matplotlib charts and tests

How to read the evidence

All results are synthetic simulation outputs. The report uses the recorded files in results/experiments.json and results/summary.csv. Units are virtual CPU cores, GiB of RAM and simulated milliseconds. No measured cloud benchmark or physical infrastructure is involved.

Team member names, student IDs and individual contributions should be added by the team before submission. No names or contributions have been assumed.

System architecture and data model

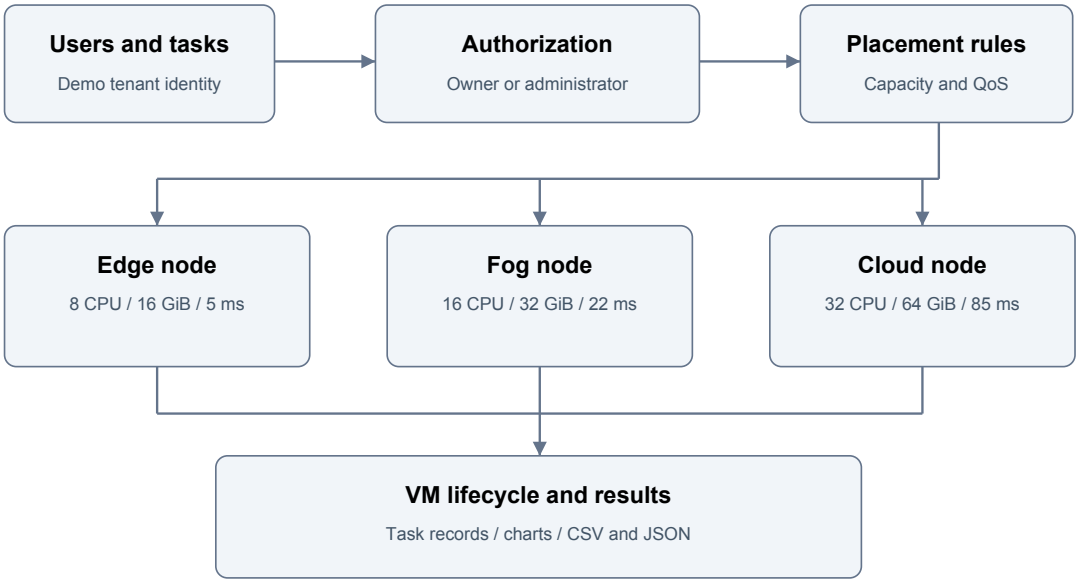


Figure 1. Tasks pass through authorization and placement checks. Admitted tasks reserve resources on one node through a VM record. Completion and stop events release reservations. Results expose permitted task records and shared node telemetry.

Node	CPU	RAM GiB	Base latency ms
Edge	8	16	5
Fog	16	32	22
Cloud	32	64	85

Module responsibilities

simulator/core.py contains validated data classes, placement, the event clock, resource accounting and authorization. **simulator/experiments.py** runs isolated experiment instances and writes evidence. **simulator/charts.py** creates Matplotlib figures. **app.py** presents controls and scoped results. **cli.py** offers demo and experiment commands.

Records and lifecycle

A task stores its ID, tenant, CPU, RAM, latency sensitivity, service duration and arrival time. A node stores total capacity and base latency. A VM stores VM ID, task ID, owner, host node, allocated CPU/RAM, status, creation time and scheduled finish time. Result records keep the selected node, policy score explanation, predicted response, budget and outcome.

An admitted task starts as running and becomes completed when the event clock reaches its finish time. Stopping its VM marks the VM stopped and the task cancelled. A rejected task creates no VM. The simulator computes current utilization from running VMs and retains peak utilization after completion.

Placement policies and QoS model

Common admission rules

For each arriving task, the simulator first completes VMs due at or before the task arrival time. It then inspects each node. A node qualifies only if its available CPU and RAM can satisfy the entire request. It calculates projected resource pressure after reservation and rejects candidates whose predicted response exceeds the task budget.

Projected pressure = max((used CPU + requested CPU) / total CPU, (used RAM + requested RAM) / total RAM).

Predicted response in ms = base node latency + task service duration + 20 x projected pressure.

Sensitivity	Response budget ms
High	100
Medium	220
Low	600

Lowest latency first

Among eligible nodes, choose the smallest base node latency. Break ties by node ID. If the closest node lacks capacity or fails the response budget, try the next eligible node. Reject the task when no candidate remains. This policy favors locality even when it concentrates reservations on Edge.

Capacity aware

Among eligible nodes, minimize **0.35 x (predicted response / task budget) + 0.65 x projected pressure**. Lower pressure receives greater weight. Break score ties by base latency, then node ID. The two weights sum to one and represent an explicit design choice rather than an optimized result.

Timing and success definitions

The model uses an event clock rather than sleeping. Each VM holds its exclusive allocation for the full simulated response interval. A task succeeds only when its VM completes. Rejections and cancellations remain separate outcomes. The displayed mean response uses admitted tasks, including those still running or later cancelled. Completed-within-budget percentage uses all submitted tasks as the denominator.

The 20 ms pressure penalty is a transparent modeling assumption. Resources remain reserved until the scheduled finish, so there is no CPU overcommit, queue, preemption or stochastic runtime failure. The report does not interpret admission checks as proof of real service-level guarantees.

Implementation and tenant security

Input validation and deterministic execution

Data classes reject missing IDs, unknown sensitivity levels, zero or negative capacities, and non-finite numeric values. Workload batches reject duplicate IDs and unauthorized ownership before placing any task. A seeded private random generator makes the same task sequence repeatable without changing global random state. Equal scores use stable tie breakers.

VM resource accounting

Submitting an admitted task creates its VM record. Resource use is the sum of allocations belonging to running VMs on a host. Stop is idempotent: stopping an already stopped VM cannot release its allocation twice. Completion considers only running VMs, so a stopped task cannot later turn into a completed task. Reset is required before changing nodes after a run to preserve meaningful peak percentages.

Authorization boundary

Alice represents team-a, Bob represents team-b and Instructor has the administrator role. Principals contain synthetic identity, tenant and role values. A tenant can submit only its own tasks, read its own records and stop its own VMs. Lists and exported JSON apply the same owner filter. Administrators can inspect all records and configure nodes. Raw utilization timelines require the administrator role, while aggregate node capacity telemetry is shared.

The interface intentionally permits demo identity switching. No passwords are stored. A local operator who can run the code can construct any principal or read exported files. These identities demonstrate authorization in a trusted local teaching application. They do not provide hardened authentication, encrypted storage or an internet-facing security boundary.

Unauthorized access experiment

The security experiment creates PRIVATE-A for Alice and PRIVATE-B for Bob. Bob requests PRIVATE-A through get_record. The model raises AccessDenied and records a denied audit entry. Bob then exports his session. The exported task and VM records include PRIVATE-B only. The experiment independently checks both direct-access denial and export isolation.

Interface and saved results

Simulation shows workload controls, node meters, a results table, Matplotlib charts and CSV/JSON downloads. Nodes and VMs shows the registry and stop control. Experiments runs fresh isolated simulations. Security demonstrates a denied lookup. Session data remains in memory until reset or server restart; downloadable exports and CLI evidence provide durable files.

Experiment method and comparison results

All experiment runs use fresh engines. E1 and E4 reuse the identical ordered 12-task workload generated with seed 305 at a burst arrival time of zero. Default nodes match the architecture table. CSV files retain the task inputs and every placement decision.

E1 Same workload with two policies

Setup. 12 tasks, seed 305, burst at t=0, default nodes; fresh engine per policy.

Expected. Capacity-aware placement may distribute tasks differently. Neither policy exceeds capacity or admits a task above its latency budget.

Actual. 10/12 placements differ. Lowest-latency completed 12; capacity-aware completed 12.

E4 Capacity and latency change

Setup. Repeat E1 lowest-latency workload after changing Edge from 8 CPU/16 GiB/5 ms to 16 CPU/32 GiB/2 ms.

Expected. Larger, faster Edge admits more tasks locally and changes response time and resource utilization.

Actual. 4/12 placements change. Mean response 69.25 ms before and 59.292 ms after. Fog peak CPU falls from 93.75% to 43.75%; Edge peak CPU remains 100% with twice the capacity.

Run	Total	Complete	Reject	Mean ms	QoS %
Latency first	12	12	0	69.25	100.00
Capacity aware	12	12	0	97.10	100.00
Overload	30	17	13	107.76	56.67
Upgraded Edge	12	12	0	59.29	100.00

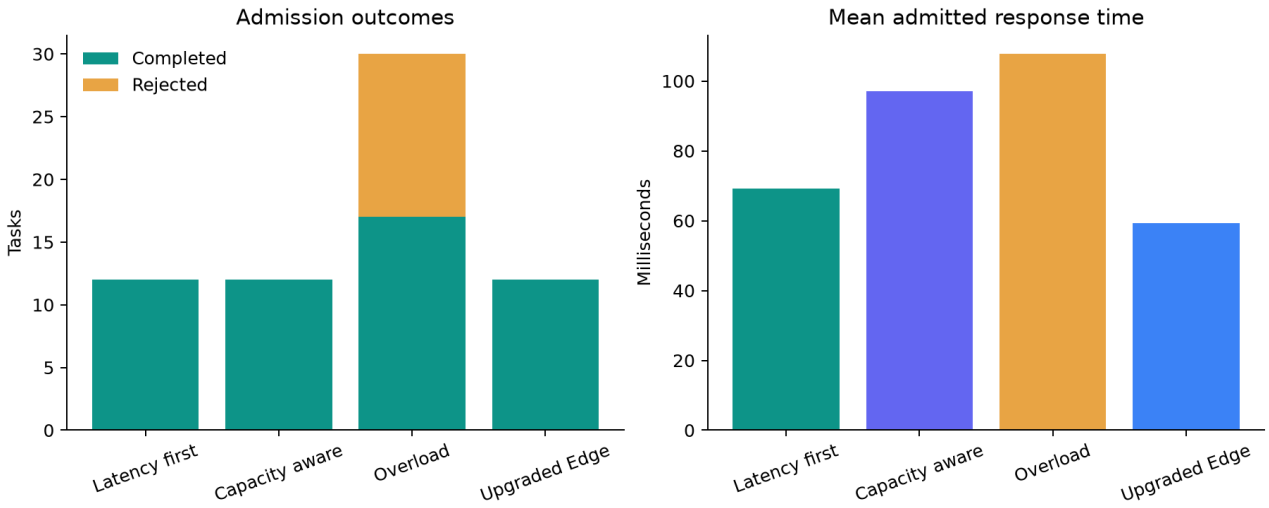


Figure 2. Mean response covers admitted tasks. QoS percentage counts completed tasks against all submitted tasks. Lower mean response does not independently describe rejected demand.

Overload evidence and resource utilization

Setup. 30 simultaneous medium-sensitivity tasks, each 3 CPU, 6 GiB RAM and 40 ms service; lowest-latency policy.

Expected. Fill Edge, then fall back to Fog and Cloud. Reject remaining tasks when no suitable capacity remains.

Actual. 17 completed and 13 rejected. Admitted tasks span 3 tiers.

Run and node	Placed	Peak CPU %	Peak RAM %
Latency / Edge	4	100.0	100.0
Latency / Fog	8	93.75	75.0
Latency / Cloud	0	0.0	0.0
Capacity / Edge	2	37.5	37.5
Capacity / Fog	4	62.5	40.62
Capacity / Cloud	6	31.25	32.81
Overload / Edge	2	75.0	75.0
Overload / Fog	5	93.75	93.75
Overload / Cloud	10	93.75	93.75

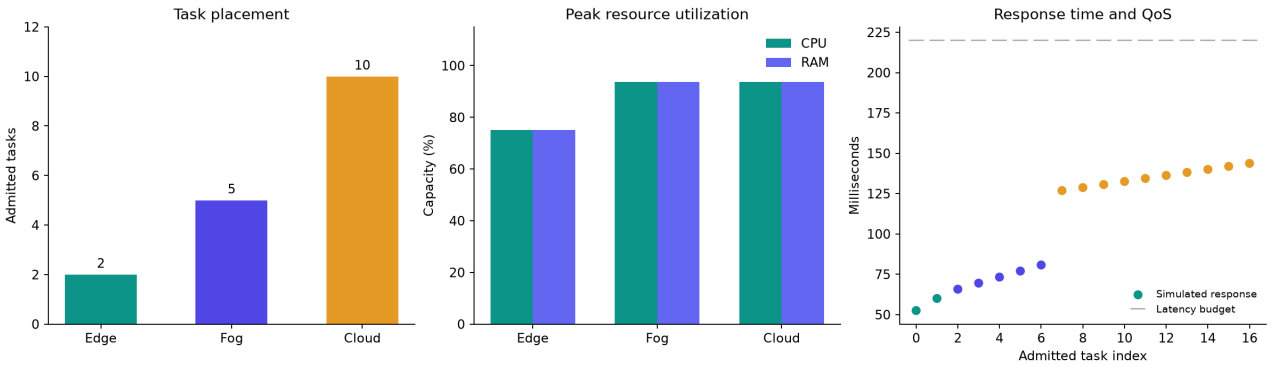


Figure 3. Overload uses 30 tasks of 3 cores and 6 GiB each. Edge admits 2, Fog admits 5 and Cloud admits 10. The remaining CPU on each host is insufficient for another 3-core request. Current use returns to zero after all admitted VMs complete.

Interpretation and evidence files

The capacity-aware policy reduces Edge peak CPU from 100% to 37.5% on this workload while increasing mean response from 69.25 ms to 97.10 ms. It does not improve completion count here. The Edge upgrade lowers mean response by about 14.4%. These are workload-specific comparisons under the stated model.

Evidence files include results/summary.csv, the per-run tasks, workload, nodes and timeline CSVs, comparison.png, overload_chart.png and security.json. The full machine-readable bundle is results/experiments.json. Each scenario includes setup, expected outcome, actual outcome, pass status and evidence filenames.

Validation limitations and demonstration

Validation evidence

The automated suite checks deterministic workloads, capacity fallback, independent RAM limits, latency-budget rejection, VM stop and completion, arrival-time reuse, duplicate IDs, batch atomicity, invalid inputs and tenant restrictions. It also runs all four experiment assertions. Streamlit AppTest exercises workload submission, VM stop, experiment execution, tenant restrictions and the security demonstration. Test evidence is saved in evidence/test-results.txt.

Limitations and extensions

The scheduler reserves exclusive resources and uses a simple linear pressure penalty. It does not model bandwidth, data locality, processing speed differences, energy, monetary cost, migration, contention queues or failures after admission. The burst workload is small and results are sensitive to task order, node configuration and chosen policy weights. Larger varied workloads and sensitivity analysis would be needed to evaluate a general scheduling strategy.

The security mechanism assumes trusted demo profiles. Durable storage, authentication and server-side sessions would be needed for untrusted users. CSV/JSON files are local exports rather than encrypted tenant storage. Unit labels describe simulated quantities and do not claim equivalence to provisioned hardware.

Short live demonstration

1. Start the app with python -m streamlit run app.py and leave the Instructor demo identity selected.
2. Generate 12 tasks with seed 305, choose a policy and run. Explain task decisions, response budgets and peak utilization.
3. Reset, clear Complete after placement and run again. Open Nodes and VMs, stop a VM and show that its allocation is released.
4. Open Experiments and run all four scenarios. Compare policy outcomes, overload rejection and the upgraded Edge configuration.
5. Open Security and run the unauthorized-access demonstration. Explain the difference between a trusted demo identity and authentication.

Reproduction and deliverables

README.md contains platform-specific setup, CLI commands, the simulation rules and the deliverable index. The ZIP includes runnable source, dependency versions, tests, architecture diagrams, this seven-page report, a six-slide editable presentation, results tables and charts, experiment evidence and interface screenshots. The team should complete its own names and contribution details before final submission.

Project specification: ARM305_Mini_Project.pdf supplied by the user. Streamlit testing reference: https://docs.streamlit.io/develop/api-reference/app-testing. All numerical results in this report come from the included simulator experiments.